A brief introduction of MATLAB

2024 Spring Math-UIC, By Xiaofeng Cai

2024 年 2 月 19 日

Contents

1 MATLAB使用步骤 **3**

2 MATLAB桌面操作系统 **4**

2.1 File主菜单项 4

2.2 MATLAB命令窗口(Command Window) 5

2.3 MATLAB工作空间（Workspace） 7

3 MATLAB表达式与变量 **8**

3.1 MATLAB常用算符 8

3.2 变量和常量 8

4 MATLAB的数组 **9**

4.1 数组的建立和保存 9

4.2 数组的运算 9

4.3 数组相关函数 9

5 MATLAB常用函数 **10**

6 MATLAB的矩阵分析与处理 **11**

6.1 常用的特殊矩阵 11

6.2 专门学科的特殊矩阵 11

6.3 用矩阵求逆方法求解线性方程组 11

7 MATLAB符号数学工具箱 **12**

7.1 创建符号对象 12

7.2 符号表达式中的自变量 12

7.2.1 自由变量的确定原则 12

7.2.2 findsym函数 12

7.3 符号矩阵 12

7.4 符号表达式的四则运算 13

7.5 基本函数 13

7.6 符号函数画图 13

8 MATLAB数据绘图快速入门 **14**
8.1 MATLAB利用绘图函数绘图 . . . 14
8.1.1 二维图形的点值画图 . . . 14
8.1.2 在图形窗中添加图形和图形的清除与重现 . . . 14
8.2 导出图形 . . . 14
8.3 数据画图实例 . . . 16

9 MATLAB编程 **19**
9.1 MATLAB的关系运算符 . . . 19
9.2 MATLAB的逻辑运算符 . . . 19
9.3 MATLAB程序中其它几个有用的命令 . . . 19
9.4 子函数和私用函数 . . . 20
9.5 inline函数与匿名函数 . . . 20
9.6 程序调试 . . . 21
9.7 编程技巧 . . . 21
9.7.1 矢量化 . . . 21
9.7.2 预分配内存空间 . . . 21

10 编程实例及作业模板 **22**
10.1 用Lagrange插值分析Runge现象 . . . 22
10.1.1 实验目的 . . . 22
10.1.2 Chebyshev多项式 . . . 22
10.1.3 实验步骤 . . . 22
10.1.4 实验结果 . . . 22
10.1.5 实验分析 . . . 23
10.1.6 计算代码 . . . 23
10.2 列选主元的Gauss消去法 . . . 27

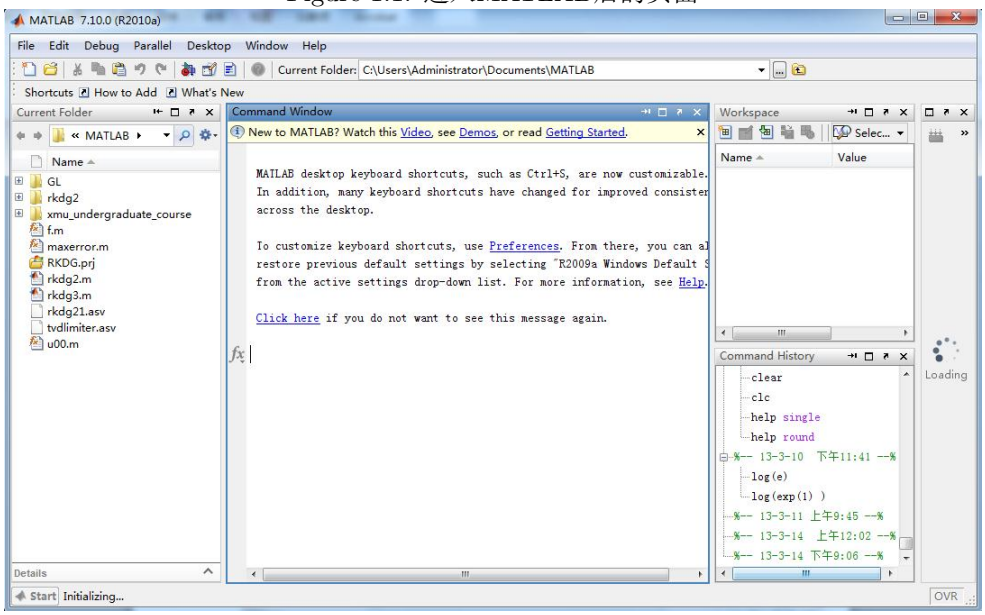
MATLAB是一个高性能的工程计算和数值计算软件，拥有功能全面的函数库。因此，正版MATLAB非常贵，当然咱们UIC有购买了Matlab的使用权。因为软件具有各种函数库，MATLAB可以看成是我们专业的高级计算器。希望大家能够通过学习计算方法课程和上机课学习到这个软件的基本用法。

关于如何学习MATLAB，我们知道很多人有一个办法，就是通过（https://cn.bing.com/?mkt=zh-CN）搜索函数用法。商业公司一定提供详细的使用说明书解释如何使用他们的商品，因此，真正最快的了解到软件，应该学习软件公司自己提供的讲义（http://www.mathworks.com/academia/student_center/tutorials/launc或者google搜索'Matlab Tutorial'，当然在使用过程中尽量使用help。

1 MATLAB使用步骤

1) 进入MATLAB后的页面

Figure 1.1: 进入MATLAB后的页面



2) 脚本文件

很多同学直接在Command Window上直接演示直接编写自己想要的调用函数过程，但是你可能要用到很多的比较，以及进行更复杂的程序编写，因此需要使用脚本**.m。

由左上角"File"—->"New" —->"Script"， 编写程序完后保存为"c1x1.m"的m文件。

注意：

(1) 每次编程之前务必加上clear（清除工作空间中的所有变量）。一般还要加clc（清楚命令窗口）。

(2) 保存文件必须有后缀".m"。并且必须用**字母**字母开头，命名成**"1.m"、"1.1.m"**是错误的。

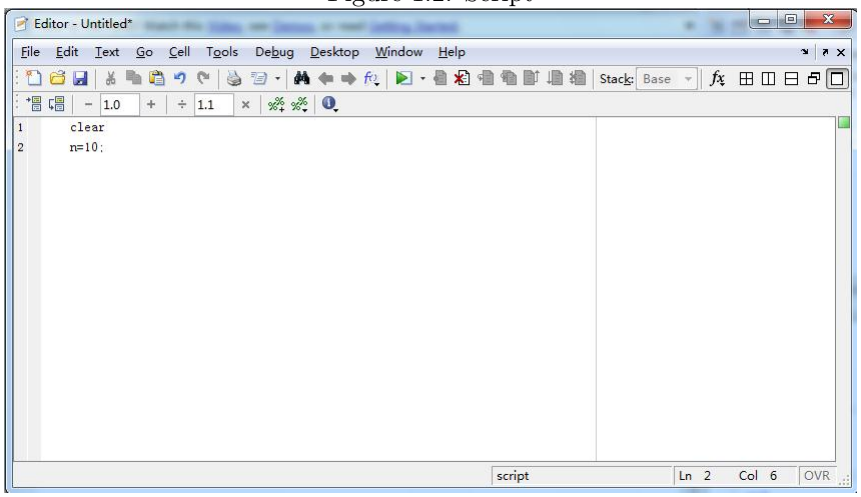
(3) 需要把保存的m文件添上自己的名字缩写以防止与matlab库函数重名，比如，您写了一个newton插值的函数且你的名字叫"白富美"，需要把文件名命名为**"newton_bfm.m"**。

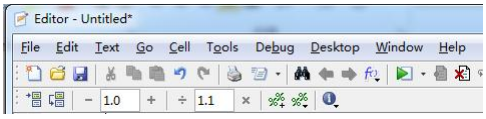
(4) 最好 **不要保存在包含中文的路径**中，这样有可能会让你的程序运行不了。

(5) 保存文件，使文件位于MATLAB能够搜索到得路径中。

3）按Window下的图标 运行程序。

Figure 1.2: Script



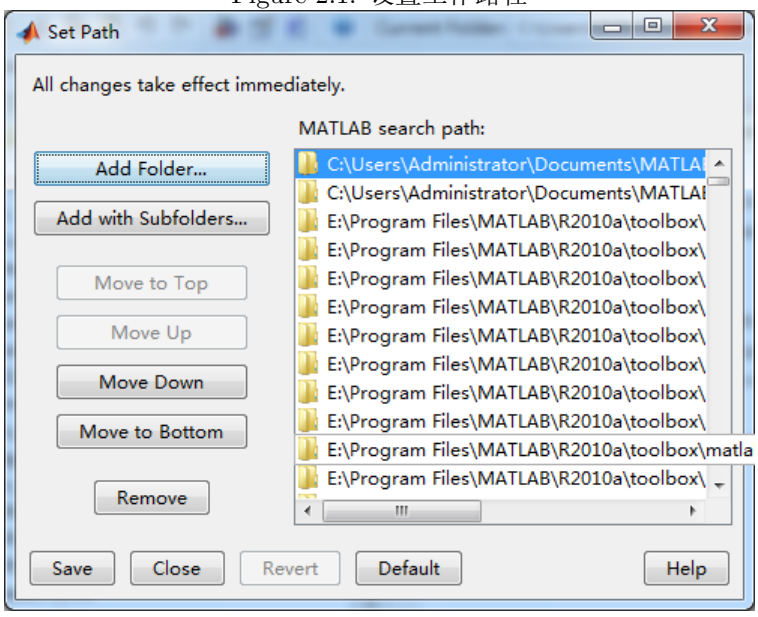


2 MATLAB桌面操作系统

2.1 File主菜单项

Save Workspace As：用于把工作空间的数据存放到相应的路径文件中。

Figure 2.1: 设置工作路径



Set Path：设置工作路径。因为MATLAB只能调用MATLAB搜索路径下的函数文件，因此如果我们想在自己的文件下运行程序，必须把这个文件夹名添加到工作路径下。

Preferences：用于设置命令的属性。例如设置长型数据类型等。

Figure 2.2: Add Folder to Path

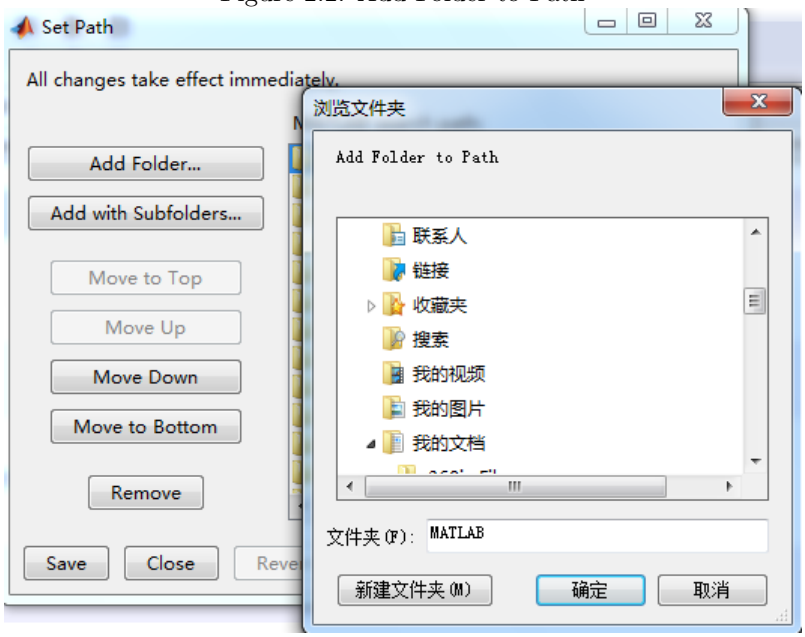
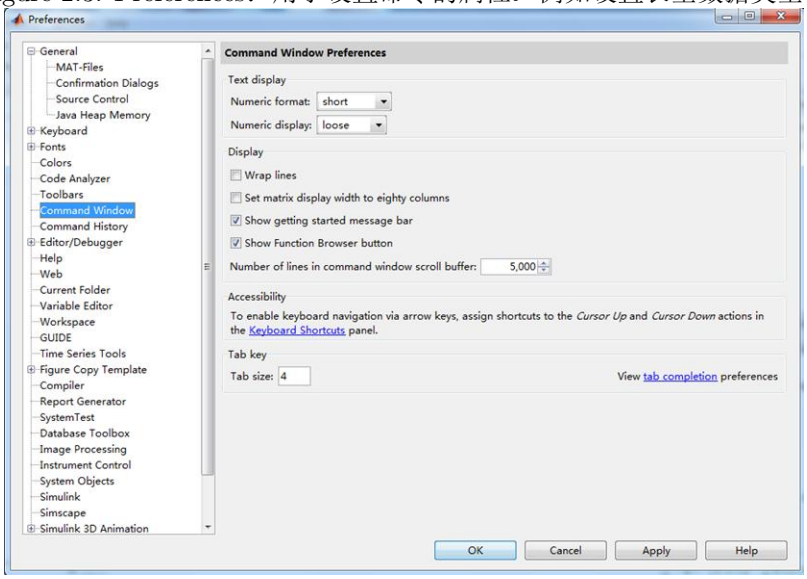


Figure 2.3: Preferences：用于设置命令的属性。例如设置长型数据类型等



2.2 MATLAB命令窗口(Command Window)

MATLAB的命令窗口如图所示，它用于MATLAB命令的交互操作，具有两大主要功能：

（1）提供用户输入命令的操作平台，用户通过该窗口输入命令和数据。

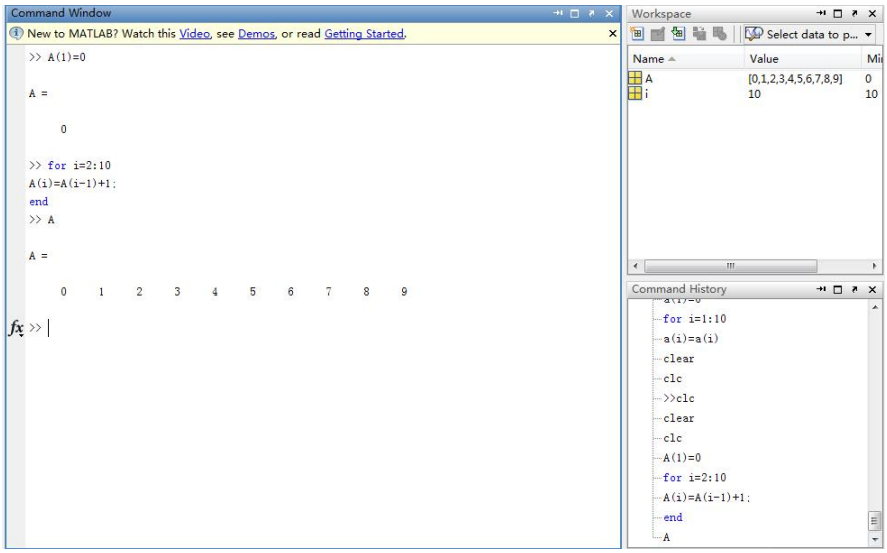
（2）提供命令执行结果的显示平台，该窗口显示命令执行的结果。

因此除了建立脚本工作外，也可以直接在命令窗口中直接输入命令语句。

MATLAB语句的形式为:

>>变量=表达式。

通过等号将表达式的值赋予变量。当键入回车键时，该语句被执行。语句执行后窗口自动显示出语句执行的结果。




```
>>  A(1)=0

A =

     0

>>  for i=2:10
A(i)=A(i-1)+1;
end
>>
>> A

A =

     0     1     2     3     4     5     6     7     8     9

>>  a=1+2+3+...
+4+5;
>> a

a =

    15

>>
```

说明:

(1) 三个小黑点是"连行号"，分号：”；”作用是指令执行结果将不显示在屏幕上，但变量a将驻留在内存中。如果”A(1)=0”不加分号，那么我们可以看到结果显示在屏幕上了。

(2) 可以看到A的维数在不断增加，因此MATLAB是一个以不必指定维数的数组作为主要数据对象的交互系统。因此在程序编写中需要人为的固定七位数，比如通过A=zeros(1,10) 定义一个一行10列的矩阵。

(3) for循环语句：

for index=start:incredement:end

statements

end

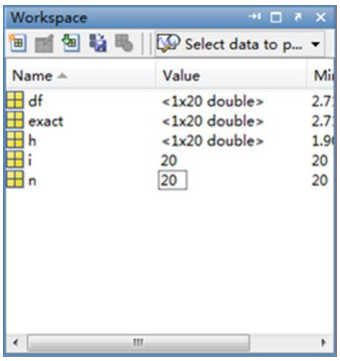
注意事项：

1.start、incredement和end可以为任意整数包括负数。但是要注意数组必须从1开始。比如”A(0)=0”和”A(-

5)=0”是不对的。

2.如果incredement为负数必须严格按照上面过程。比如”8:1”是不对的，而”8:-1:1”才是正确的。

2.3 MATLAB工作空间（Workspace）



空间常用命令：

clear：清除工作空间中的所有变量。

clear 变量名：清除指定的变量。

clf: 清除图形窗口内容。

3 MATLAB表达式与变量

3.1 MATLAB常用算符

+ 加法 - 减法 ^ 幂 * 乘法 / 右除 \ 左除

3.2 变量和常量

在MATLAB中输入"n=10"。与其他高级语言不同，"n"无需事先定义，本身是默认双精度浮点数，但同时可做"整数"使用。

1）MATLAB变量命名规则

（1）变量名区分大小写。

（2）变量名以英文字母开始，第一个字母后可以使用字母、数字、下划线，但不能使用空格和标点符号。变量名长度不得超过31位。

（3）某些常量可以作为变量使用，例如i在MATLAB中表示虚数单位，也可以作为变量使用。注意"2i+1"表示的是一个虚数，而"2*i+1"表示的是实数。

Table 3.1: MATLAB的永久常数

名称	说明	名称	说明
pi	圆周率（3.1415926...）	eps	系统的浮点（Floating-point）精确度
INF(或inf)	无限大，例如1/0	i(或j)	虚数单位，定义为$\sqrt{1}$
nan或NaN	非数值（Not a number），例如0/0	nargin	函数实际输入参数的个数
realmax	系统所能表示的最大数值	realmin	系统所能表示的最小数值

4 MATLAB的数组

4.1 数组的建立和保存

在MATLAB中数组可以看作是行向量，即只有一行的矩阵。

例如创建等差数列

x=-5:0.01:5; %生成步长为0.01的从-5到5的数组

y=linspace(-5,5,1001); %功能同上

4.2 数组的运算

两个数组的运算是将对应元素作运算得到一个新的数组。一个数与一个数组的运算是该数逐个与数组元素作运算得到一个数组。

数组运算符+，－, ".*", ". /"，". \", ".^"。

注意两个同阶矩阵A,B的对应元素之间（不是整个矩阵）也可以作运算，使用特殊的"."表示，得到同阶的矩阵C，因此数与数之间的运算也可以用".",如

$C = A.*B$,其中$C(i,j) = A(i,j)*B(i,j)$;

$C = A. \wedge B$,其中$C(i,j) = A(i,j) \wedge B(i,j)$;

$C = A./B$,其中$C(i,j) = A(i,j)/B(i,j)$;

$C = A. \setminus B$,其中$C(i,j) = A(i,j) \setminus B(i,j)$;

4.3 数组相关函数

Table 4.1: 适用于数组的常用函数

函数名	功能	函数名	功能
min(x)	数组x的元素的最小值	max(x)	数组x的元素的最大值
mean(x)	数组x的元素的平均值	median(x)	数组x的元素的中位数
diff(x)	数组x的相邻元素的差	sort(x)	对数组x的元素进行排序（Sorting）
length(x)	数组x的元素个数	norm(x)	数组x的欧氏（Euclidean）长度
sum(x)	数组x的元素总和	prod(x)	数组x的元素总乘积
dot(x, y)	数组x和y的内积	cross(x, y)	数组x和y的外积

当然Matlab提供了大量的数学函数库，因此你会遇到很多不认识的函数，或者你想使用某种功能的函数，你可以使用help指令。

5 MATLAB常用函数

Table 5.1: MATLAB常用的基本数学函数

函数名	功能	函数名	功能
abs(x)	纯量的绝对值或向量的长度	sqrt(x)	开平方
real(z)	复数z的实部	imag(z)	复数z的虚部
conj(z)	复数z的共轭复数	round(x)	四舍五入至最近整数
fix(x)	无论正负，舍去小数至最近整数	floor(x)	地板函数，即舍去正小数至最近整数
ceil(x)	天花板函数，即加入正小数至最近整数	rat(x)	将实数x化为分数表示
rats(x)	将实数x化为多项分数展开	sign(x)	符号函数(Signum function) 当x<0时，sign(x)=-1； 当x=0时，sign(x)=0; 当x>0时，sign(x)=1。
rem(x,y)	求x除以y的余数	gcd(x,y)	整数x和y的最大公因数
lcm(x,y)	整数x和y的最小公倍数	exp(x)	自然指数
pow2(x)	2的指数	log(x)	以e为底的对数，即自然对数
log2(x)	以2为底的对数	log10(x)	以10为底的对数

Table 5.2: MATLAB常用的三角函数

函数名	功能	函数名	功能
$\sin(x)$	正弦函数	$\cos(x)$	余弦函数
$\tan(x)$	正切函数	asin(x)	反正弦函数
acos(x)	反余弦函数	atan(x)	反正切函数

6 MATLAB的矩阵分析与处理

6.1 常用的特殊矩阵

zeros:产生全0矩阵（零矩阵）；ones: 产生全1矩阵（幺矩阵）；eye: 产生单位矩阵；

rand: 产生0-1间均匀分布的随机矩阵；

randn: 产生均值为0，方差为1的标准正态分布随机矩阵；

zeros(m):产生m×m零矩阵；zeros(m,n):产生m×n零矩阵；

zeros(size(A)):产生与矩阵A同样大小的零矩阵；

6.2 专门学科的特殊矩阵

魔方矩阵：每行、每列及两条对角线上的元素之和都相等；magic(n),生成一个n阶魔方矩阵；

范德蒙德矩阵：最后一列全为1，倒数第2列为一个指定向量，其他各列是其后列与倒数第2列的点乘积；函数vander(V)生成以向量V为基础向量的范德蒙德矩阵；

希尔伯特矩阵:$h_{ij} = \frac{1}{i+j-1}$；生成希尔伯特矩阵的函数是hilb(n).

特普利茨矩阵：除第一行和第一列外，其他每个元素都与左上角元素相同。toeplitz(x,y)生成一个以x为第一列，y为第一行的特普利茨矩阵。

帕斯卡矩阵：由杨辉三角形组成的矩阵称为帕斯卡矩阵，由pascal(n)生成一个n阶帕斯卡矩阵。

6.3 用矩阵求逆方法求解线性方程组

$Ax = b$, 则$x = A^{-1}b$. 所以x=inv(A)*b; 也可以用左除运算符”\” 求解线性代数方程组，x=A\b.

7 MATLAB符号数学工具箱

符号计算仅用于公式推导（可用专门的maple软件推导），在实际编程过程中用数组形式书写程序。

7.1 创建符号对象

MATLAB提供了两个建立符号对象的函数：sym和syms。

（1）sym函数

sym函数用来建立单个符号量，一般调用格式为：

符号量名=sym('符号字符串')

（2）syms函数

函数sym一次只能定义一个符号变量，而syms一次可以定义多个符号变量。

syms函数的一般调用格式为：

syms 符号变量名1 符号变量名2 ... 符号变量名n

注意变量间用空格而不用逗号分隔。

- 实例

```
U=sym('3*x^2+5*y+2*x*y+6');                    %定义符号表达式
syms   x   y;                                  %建立符号变量x、y
V=3*x^2+5*y+2*x*y+6;                           %定义符号表达式V
2*U-V+6;                                       %求符号表达式的值
```

7.2 符号表达式中的自变量

7.2.1 自由变量的确定原则

- 小写字母i和j不能作为自由变量。
- 符号表达式中如果有多个字符变量，则按照以下顺序选择自由变量：首先选择x作为自由变量；如果没有x，则选择在字母顺序中最接近x的字符变量；如果与x相同距离，则在x后面的优先。
- 大写字母比所有的小写字母都靠后。

7.2.2 findsym函数

- 如果不确定符号表达式中的自由符号变量，可以用findsym函数来自动确定。
- **语法：**

```
findsym(f,n) % 确定自由符号变量
```

说明：f可以是符号表达式或符号矩阵；n为按顺序得出符号变量的个数，当n省略时，则不按顺序得出f中所有的符号变量。

7.3 符号矩阵

通过sym函数创立符号矩阵，矩阵元素之间可用空格或逗号分隔。

```
A=sym('[a,2*b;3*a,0]')
```

输出结果：

```
A=
[a,2*b]
[3*a,0]
```

需要注意的是：符号矩阵的每一行的两端都有方括号，这是与MATLAB数值矩阵的一个重要区别。

在MATLAB中，数值矩阵不能直接参与符号运算，必须先转化为符号矩阵。

（1）将数值矩阵转化为符号矩阵

函数调用格式为：sym(数值矩阵)

（2）将符号矩阵转化为数值矩阵

函数调用格式：numeric(A)

transpose(s):返回s矩阵的转置矩阵。

determ(s):返回s矩阵的行列式的值。

许多数值矩阵的函数可直接应用于符号矩阵。如diag、triu、tril、inv、rank、eig。

7.4 符号表达式的四则运算

- factor(S):对S分解因式，S是符号表达式或符号矩阵。
- expand(S):对S进行展开，S是符号表达式或符号矩阵。
- collect(S):对S合并同类项，S是符号表达式或符号矩阵。
- collect(S,v):对S按变量v合并同类项，S是符号表达式或符号矩阵。
- simplify(S):应用函数规则对S进行化简。
- simple(S):调用MATLAB的其他函数对表达式进行综合化简，并显示化简过程。

7.5 基本函数

微分

```
>>syms  a   x;                    %定义符号变量a和x
>>f=sin(a*x);                     %创建函数f
>>dfx=diff(f,x);                  %对x求导
>>dfa=diff(f,a);                  %对a求导
```

积分

```
>>syms  x;                        %定义符号变量a和x
>>f=x*log*(1+x);                  %创建函数f
>>int1=int(f,x);                  %对x积分
>>int2=int(f,x,0,1);              %求[0,1]区间上的积分
```

给出数值型符号结果

在符号计算中可以通过vpa()函数把符号变量以任意精度显示出来。

```
>>vpa(pi,20);                     %显示20位有效数字的π.
```

代入(substitution)

subs(s, char,value)　用符号或数值变量value替换s中的符号变量char

例如：

```
syms x; f=x; subs(f,x,1);
```

inline字符串求值

```
f=inline('5*sin(x)+3*cos(y)','x','y')
```

7.6 符号函数画图

ezplot函数画图

ezplot(f)　　在默认区间-2*pi¡x¡2*pi绘制f=f(x)的函数图

ezplot(f,[a,b])　　在区间a¡x¡b上绘制f=f(x)的函数图

例如：ezplot($'t-3'$, $[0,3]$)，在区间[0,3]上的t-3的函数。

8 MATLAB数据绘图快速入门

先介绍写基本画图的命令和过程，再给出一个例子说明。

8.1 MATLAB利用绘图函数绘图

8.1.1 二维图形的点值画图

1. 二维图形的基本命令是：

plot(x1,y1,option1,x2,y2,option2,⋯)

其中，x1与y1给出的数据分别为x轴与y轴坐标值，option1为选项参数，以做点连折线的方式绘制第一个二维图形；同时类似地绘制第二个二维图形。

这是一个最基本的命令，因为一旦画出图来，我们可通过导出的图形的界面对图形进行调整。

2. 我们先利用figure命令来创建一个新的空白图像

>>figure

>>x = 0:pi/3:2*pi;

>>y = sin(x);

>>plot(x,y)

3. 图像画好之后，可以为x轴和y轴自定义一个标签，标签就是一个坐标轴的一个名称。

>>xlabel('x')

>>ylabel('sin(x)')

4. 用“box off”去掉边框。

5. 定义指标范围：“axis([xmin,xmax,ymin,max])”

其他函数见见下表

函数名	功能
plot	x轴和y轴均为线性刻度（Linear scale）
loglog	x轴和y轴均为对数刻度（Logarithmic scale）
semilogx	x轴为对数刻度，y轴为线性刻度
semilogy	x轴为线性刻度，y轴为对数刻度

6. 线对象的常用属性

LineWidth －线条宽度

MarkerEdgeColor －点边界色彩

MarkerFaceColor －点域色彩

MarkerSize －数据点的大小

7. 图形的裁剪处理NaN常数可以用于表示那些不可使用的数据，利用这个特性，可以将图形中需要裁剪部分对应的函数值设置成NaN，这样在绘制图形时，函数值为NaN的部分将不显示出来，从而达到裁剪图形的目的。

8.1.2 在图形窗中添加图形和图形的清除与重现

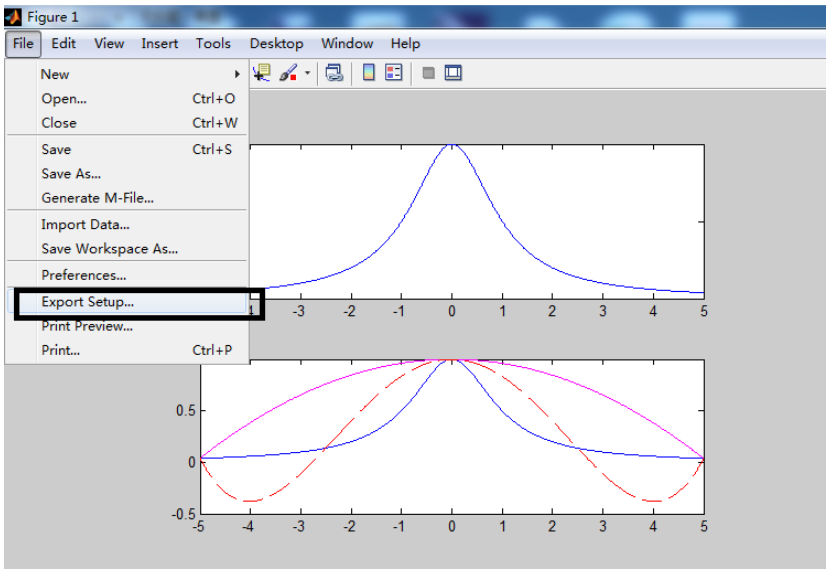
若当前的图形窗中已绘有图形，再用绘图命令作图就会将原来的图形删除。若要保留原来的图形，把新图形加上去必须用hold on 命令，而hold off 是中止这一功能的命令（此为缺省状态）。命令clf 清除当前图形窗中的图形。

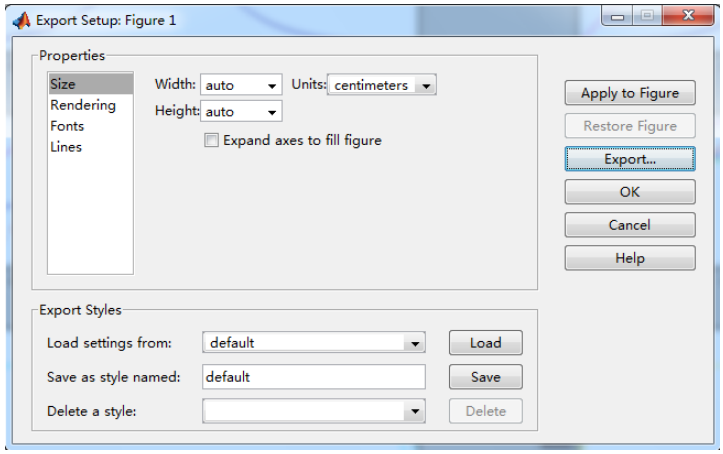
8.2 导出图形

1. 图形界面的图形导出：

Table 8.1: plot绘图函数的叁数

option值	图形表现	option值	图形表现
s	小方块	p	五角星
d	菱形	h	六角星
o	圆圈	v	向下三角形
∧	向上三角形	>	向右三角形
<	向左三角形	+	十字
*	米字	x	叉符
r	红色线	y	黄色线
k	黑色线	w	白色线
b	蓝色线	g	绿色线
m	锰紫色线	c	亮青色线
.	点	-	实线
:	点线	-.	点虚线
–	虚线		

点中export setup。 

然后在下图点中export即可导出图形。 

2. 建议直接用print命令导出图片

Table 8.2: print的命令

图像格式	命令
24位bmp	-dbmp
黑白eps	-deps
彩色eps	-depsc
黑白eps Level 2	-deps2
彩色eps Level 2	-depsc2
24位jpeg	-djpeg
彩色pdf	-dpdf
png	-dpng

8.3 数据画图实例

比如对某三个文件（bur_exact.txt,burgers_multi_shock.txt,burgers_rk_shock.txt,注意数据文件的行列与matlab矩阵对应）的数据进行分析。

用load命令读取数据文件。

>>load bur_exact.txt

burgers_rk_shock.txt

```
0.000000    0.208652    0.201842
0.200000    0.451547    0.451335
0.400000    0.702040    0.702094
0.600000    0.946489    0.947400
0.800000    1.188163    1.178432
1.000000    1.358132    1.378814
1.200000    0.739760    1.498796
1.400000   -0.279688   -0.440524
1.600000   -0.266153   -0.261093
1.800000   -0.042163   -0.039212
2.000000    0.201873    0.201842
```

burgers_multi_shock.txt

```
0.000000    0.207994    0.201842
0.200000    0.450942    0.451335
0.400000    0.702010    0.702094
0.600000    0.948631    0.947400
0.800000    1.187689    1.178432
1.000000    1.351956    1.378814
1.200000    0.754790    1.498796
1.400000   -0.288836   -0.440524
1.600000   -0.266584   -0.261093
1.800000   -0.042256   -0.039212
2.000000    0.201658    0.201842
```

bur_exact

```
0.000000    0.207994    0.201842
0.200000    0.450942    0.451335
```

```
0.400000         0.702010         0.702094
0.600000         0.948631         0.947400
0.800000         1.187689         1.178432
1.000000         1.351956         1.378814
1.200000         0.754790         1.498796
1.400000        -0.288836        -0.440524
1.600000        -0.266584        -0.261093
1.800000        -0.042256        -0.039212
2.000000         0.201658         0.201842
```

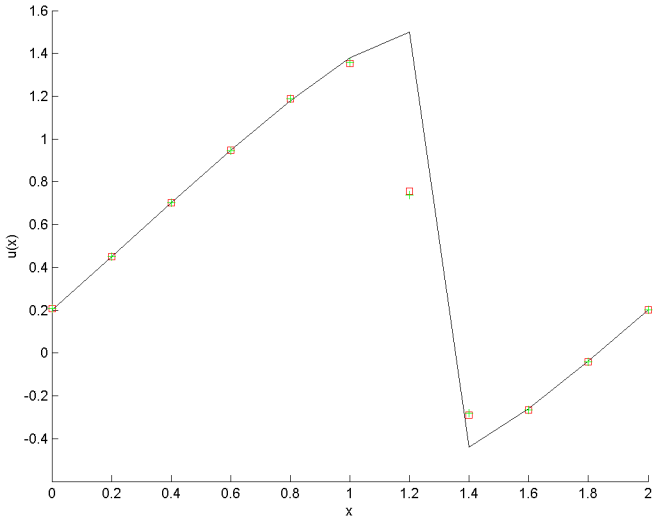
plot_f.m

```
clear
clc
figure % 创建一个新的空白图像
load burgers_multi_shock.txt  % 导入burgers_multi_shock.txt的数据
load burgers_rk_shock.txt
load bur_exact.txt

x = burgers_multi_shock(:,1);  % burgers_multi_shock.plt的第一列为数组x
y = burgers_multi_shock(:,2);  % burgers_multi_shock.plt的第二列为数组y
x1= bur_exact(:,1);
y1= bur_exact(:,3);
y2= burgers_rk_shock(:,2);



plot(x,y,'rs')
hold on
plot(x1,y1,'k')
hold on
plot(x,y2,'g+')
axis([0,2,-0.6,1.6])   % 自定义坐标范围
box off          % 去掉边框
xlabel('x','FontSize', 24)      % x轴的标签
ylabel('u(x)','FontSize', 24)   % y轴的标签
print -dpng bur10.png %导出png图片
```



结果如下图：

9 MATLAB编程

1. 1966年，Bohm和Jacopini提出了三种基本的程序设计结构：顺序结构、选择结构、循环结构。已经证明，这三种结构组成的算法，可以解决任何复杂问题。这三种结构都有四个特点：

（1）只有一个入口。

（2）只有一个出口。

（3）结构内不存在死循环。

（4）结构内的每一部分都有机会被执行到。

2. **分行和注释**

(1) 在MATLAB中，同一个物理行允许有多个MATLAB的命令或语句，命令之间用逗号或分号分隔。

(2) 用几个物理行来表示一个命令行可用续行符号”…”。

(3) 百分号”%”是MATLAB的注释行命令，它必须处于第一个字符。MATLAB不对其作任何实际的操作。

3. **用Ctrl+C停止运行**

9.1 MATLAB的关系运算符

== 等于 ~= 不等于

9.2 MATLAB的逻辑运算符

& 逻辑与 | 逻辑或 ~ 逻辑非

9.3 MATLAB程序中其它几个有用的命令

全局变量：

MATLAB函数M文件中的变量是局部的。一旦退出程序返回MATLAB系统，程序中出现的中间变量不复存在。若在退出程序之后还需用到这些变量，必须将它们设置为全局变量。可以在函数M文件建立时，用global命令定义全局变量。

格式为：global (<变量名集>)

将<变量名集>中的每个变量都设置为全局变量。在该文件执行完毕后，这些变量仍在工作空间内，全局变量的说明应出现在使用它们之前。

键盘输入数据和字符串

键盘输入数据的命令是：input (<提示>)

用法为<变量> = input (<提示>)

其中<提示>是一字符串。遇到键盘输入命令时，系统显示提示字符串且暂停等待输入，输入以回车结束，系统将输入的内容赋值给<变量>指明的变量，输入对象可以是数、矩阵或数组。

命令: <字符变量> = input (<提示> , ′ s′)

用来从键盘输入字符串，注意输入的字符串应用单引号括起来。

格式输出

disp (<字符串>)

可将<字符串>内容原封不动地显示出来；

disp (<矩阵名>)

可将<矩阵名>指明的矩阵变量的内容显示出来（不显示矩阵变量名）。

数据的格式输出命令为fprintf (<格式> , <变量集>)

其功能是将<变量集>指明的变量用<格式>指明的格式显示。

<格式>是一个字符串，主要有

′ %m.ne′ （用m位显示数的科学表示，小数位数为n位）

′ %m.nf′ （用16位数浮点表示显示，小数位数为n位）。

eval

若s是一个字符串，其内容为一个命令或表达式，那么命令eval (s) 即将此字符串转化为命令或表达式加以执行或进行运算。例如：

```
s='1/ ( i + j -1 )';
for i = 1:3
for j=1:3
A(i,j)=eval ( s );
end
end
```

该程序段生成一个三阶Hilbert阵。

feval

feval指令的基本语法格式为：

[y1,y2,⋯]=feval('FUN',arg1,arg2,⋯)

其中，参数FUN为函数名，arg1,arg2,⋯分别为输入参数，y1,y2,⋯存储计算结果。FUN字符串仅为函数名，输入参数由arg1,arg2,⋯给出。eval()和feval()通常可以完成同样的计算过程，但是它们的使用格式有所不同。例如，下面使用eval()和feval()的指令都能够完成相同的操作。

```
eval('s=cos(pi/2)')
s=eval('cos(pi/2)')
s=feval('cos',pi/2)
```

输出结果为：

```
s =
6.1232e-017
```

9.4 子函数和私用函数

在MATLAB中，一个M文件函数中只包含一个主函数，但是可以包含多个子函数，这样的子函数只能专属于这一个M函数文件，故又称之为私用函数，例如，下面的例子给出了多个子函数的情况。

```
function [max,min]=fun(x)
n=length(x);
max=fun1(x,n);
min=fun2(x);
function r=fun1(x,n)
x1=sort(x);
r=x1(n);
function r=fun2(x)
x1=sort(x);
r=x1(1);
```

其中，fun(x)为主函数，fun1(x)和fun2(x)为子函数，值得注意的是，fun1(x)和fun2(x)只能被主函数fun(x)调用。

9.5 inline函数与匿名函数

$f(x,y)=\sin(x^2+y^2)$可以用f=inline('sin(x.^2+y.^2)','x','y')直接定义。

匿名函数是MATLAB 7.0版提出的一种全新的函数描述形式。例如，若在MATLAB工作空间内已经定义了a和b变量，则匿名函数可以用f=@(x,y)a*x.^2+b*y.^2的格式定义数学关系式$f(x,y)=ax^2+by^2$，这样无需将a,b作为附加参数在输入变量里表示出来，所以使得数学函数的定义更加方便。

注意，在匿名函数定义时，a, b的值以当前MATLAB工作空间中的数值为主，在使用@定义匿名函数后，a, b的值再发生变化，则在函数中的值将不随着改变。从执行效率看，匿名函数比inline()速度快得多，而从功能上看，匿名函数可以实现inline()函数的全部功能。所以在实际应用中可以直接采用匿名函数，而不采用inline()函数。

9.6 程序调试

很多同学很快就写完了程序，然后程序中总是出现问题，那么这个程序无法真正的完成。因此写程序的功夫应该是在调试程序上。每个都可能总结出自己的调试方法。我这里简要说明如何调试。

首先，你要充分理解你要所要编写的数值方法过程，建议细致的写下程序流程图。

我们知道我们所写的程序是一个具有某种功能的数值方法的实现，因此，我们可用一些课本后面可精确手算的结果检验程序或者用于调试。

调试过程常用到的命令是**pause**，或者Debug里面设置断点。

9.7 编程技巧

对于我们这些在开始学习使用MATLAB编程的菜鸟时，我们往往只是追求作业的完成，我们会发现不同人写的程序的运行效果不一样。这里简要给出一些编程技巧。

9.7.1 矢量化

矢量化可以使得运行加快。

例子：

```
x = 0.1;
for    k=1:1001
       y(k)=log10(x);
       x=x+0.1;
end
```

进行矢量化后如下所示：

```
x= 0.1:0.1:10;
y=log10(x);
```

9.7.2 预分配内存空间

```
A = zeros(1,10);
A(1) =0;
for    n=2:10
       A(n)=A(n-1)+1;
end
```

如果没有内存预分配，每循环一次，MATLAB解释器就会是A维数加一。而内存预分配后，就取消了这一步骤，从而使运行加速。

10 编程实例及作业模板

10.1 用Lagrange插值分析Runge现象

请考虑如下问题：

（1）给定$(x_i, y_i), i = 0, 1, ..., n$,分别用Lagrange插值方法，Neville方法，Newton方法，编制程序计算插值多项式.

（2）给定$f(x) = \ln x$的数值表，取插值节点分别为$0.4, 0.5, \ldots, 0.9$和对应的函数真值。请计算$\ln 0.52$的近似值：

(a) 以0.5,0.6为节点计算.

(b) 以$\{0.5, 0.6, 0.7\}$为节点,以$\{0.4, 0.5, 0.6\}$为节点.

（3）检验Runge效应,检验教科书的内容：（a）等距节点；（b）切比雪夫点.

10.1.1 实验目的

给定$n+1$个节点及在这些节点相应的函数值，我们利用拉格朗日插值得到n次多项式，使得这个多项式通过即插值这$n+1$个函数值。观察随着n的增大，插值函数$P_n(x)$与被插值函数的逼近程度。

10.1.2 Chebyshev多项式

一般形式为

$$T_n(x) = \cos(n \arccos x), \quad |x| \leqslant 1. \tag{10.1}$$

令$T_n(x) = 0$,则

$$\begin{aligned} & T_n(x) = \cos(n \arccos x) = 0, \\ \Longrightarrow & n \arccos x = \frac{\pi}{2} + k\pi, \\ \Longrightarrow & \arccos x = \frac{(2k+1)\pi}{2n}, \\ \Longrightarrow & x_k = \cos \frac{(2k+1)\pi}{2n}. \end{aligned}$$

则$x_k = \cos \frac{(2k+1)\pi}{2n}$为切比雪夫点。

10.1.3 实验步骤

取被插值函数为

$$f(x) = \frac{1}{1+x^2}, x \in [-5, 5]$$

(1)插值节点为等距节点$x_i = -5 + \frac{10i}{n}, \quad i = 0, 1, \ldots, n$。其中$n$取$2^j, j = 1, 2, 3, 4, 5$。

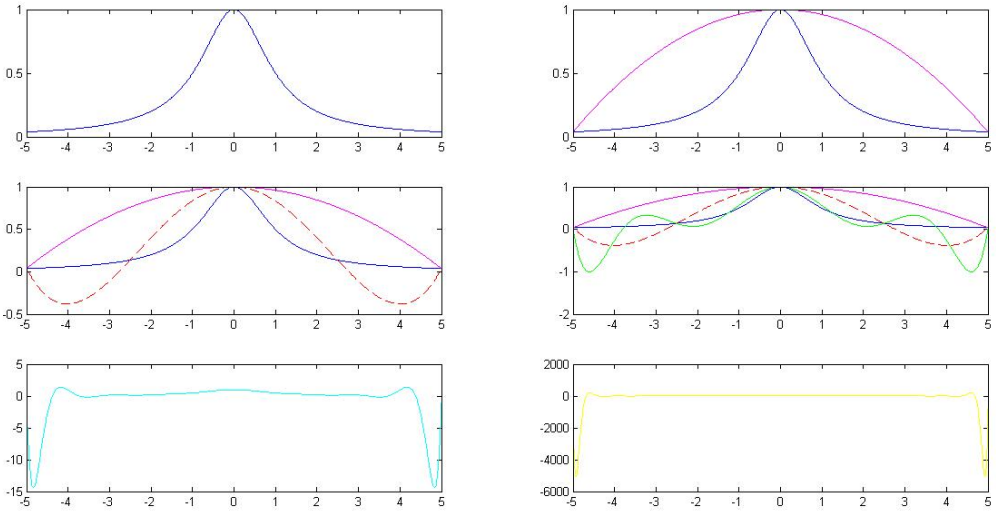
(2)插值节点为切比雪夫节点。

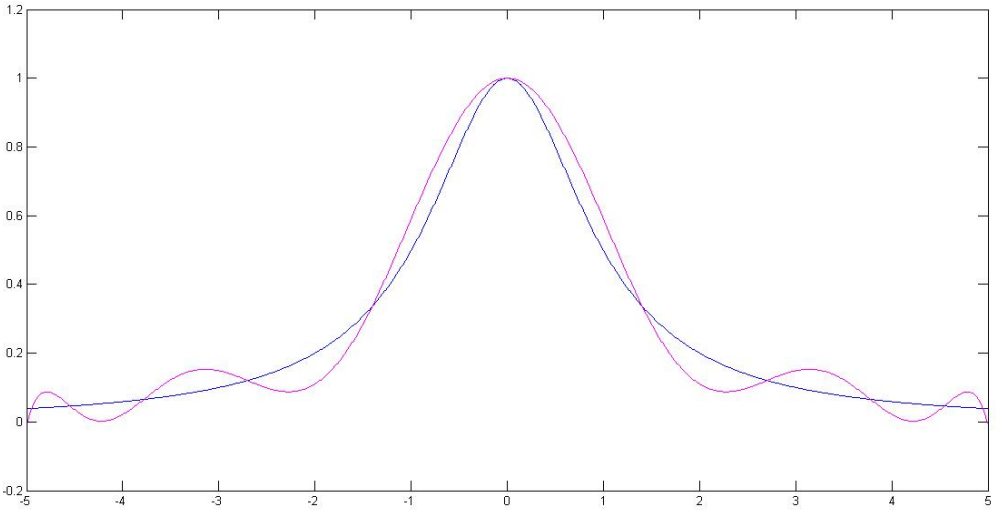
作出原始图像与各个n次插值逼近函数的图像进行比较，观察逼近情况。作图时要求在区间[-5，5]等距取1001个点，即1000等分。

10.1.4 实验结果

用Matlab6.5绘制等距Lagrange多项式插值.等距节点插值多项式的情况见下图.

取切比雪夫节点插值多项式的情况见下图





10.1.5 实验分析

从图中看出,在自变量靠近-5或5时余项会随着n的增大而很大,y5(15)=-3.4879e+003,而$f(x) > 0$. 从图中还可看出在0点附近插值效果是好的,即余项较小.另一个现象是插值多项式随节点增多而振动更多. 这种插值多项式当节点增加时反而不能更好地接近被插函数的现象称为Runge现象.

然而我们选择Chebyshev节点插值是具有好的逼近效果。

Neville逐步插值应该比Lagrange插值更优越。对Lagrange插值来说，临时增加一个节点，需全部重新计算；而Neville逐步插值仅需在原计算基础上作某种修正即可。

10.1.6 计算代码

c2x1lagrange.m

```
clear  %清除工作空间(Workspace)中的所有变量,
clc    %擦去一页命令窗口，光标回屏幕左上角
```

```
clf    %清除图形窗口内容
%以上命令为matlab开头命令
x=-5:0.01:5;    %生成步长为0.01的从-5到5的数组
y=1./(1+x.^2);  %结果数组运算生成一组以y=1/(1+x^2)对应的数组，注意算符“./”
subplot(3,2,1); %3行2列子图的第1个位置
plot(x,y,'-b'); %画x，y分别为横纵坐标的蓝色实线（-）
%n=2
x1=[-5:5:5]; f1=1./(1+x1.^2);   %matlab可用分号同行输入，用...连行
y1=lagrange(x1,f1,x); %调用函数lagrange，其中f1对应x1插值节点值，x为近似点，y1近似点值
subplot(3,2,2); %3行2列子图的第2个位置
plot(x,y,'-b'); %画x，y分别为横纵坐标的绿色实线（-）
hold on         %使当前图形与坐标轴保持原状态，不被刷新，即图形叠加。
plot(x,y1,'-m');
%n=4
x2=[-5:10/4:5];f2=1./(1+x2.^2);
y2=lagrange(x2,f2,x);
subplot(3,2,3)
plot(x,y,'-b')
hold on
plot(x,y1,'-m')
plot(x,y2,'--r')
%n=8
x3=[-5:10/8:5];f3=1./(1+x3.^2);
y3=lagrange(x3,f3,x);
subplot(3,2,4)
plot(x,y,'-b')
hold on
plot(x,y1,'-m')
plot(x,y2,'--r')
plot(x,y3,'-g')
%n=16
x4=[-5:10/16:5];f4=1./(1+x4.^2);
y4=lagrange(x4,f4,x);
subplot(3,2,5)
plot(x,y4,'-c')
%n=32
x5=[-5:10/32:5];f5=1./(1+x5.^2);
y5=lagrange(x5,f5,x);
subplot(3,2,6)
plot(x,y5,'-y')
```

lagrange.m

```
%function 输出值= 函数名( 输入值 )
function y=lagrange(x0,y0,x)
%函数功能:
%函数为lagrange插值函数
%输入值: x0为插值点，y0为插值点值，x为近似点，y为近似点值
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
n=length(x0);m=length(x); %length为数组个数
%for循环结构
for i=1:m        %近似点循环
    z=x(i);      %赋值
    s=0;         %求和程序赋0初值
    for k=1:n    %Lagrange插值开始
        p=1;             %连乘程序赋1值开始
        for j=1:n        %连乘for循环
            if j~=k      %if选择结构：j不等于k时执行下面过程
                p=p*(z-x0(j))/(x0(k)-x0(j));  %插值基函数
            end
        end
        s=p*y0(k)+s;     %基函数乘以点值相加循环
    end
    y(i)=s;              %得到近似值y
end     %for循环结束
end     %函数结束
```

neville.m

```
function y=neville(x0,y0,x)
%函数功能:
%函数为Neville插值函数
%输入值：x0为插值点，y0为插值点值，x为近似点，y为近似点值
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
n=length(x);  %n个近似点
m=length(x0);
for i=1:n
    % 编程过程:
    % p=( y0(0)*( x(i) -x0(1) )  -  y0(1)*( x(i) -x0(0) ) )...  %行列式
    % /( x(0) -x(1) );             %抄课本
    %尝试编写
    %     for j=1:m
    %         p(j+1)= ( y0(1)*( x(i) -x0(j+1) )  -  y0(j+1)*( x(i) -x0(1) ) )...  %行列式
    %             /( x(1) -x(j+1) );
    %     end
    %     for j=2:m
    %         q(j+1)=( p(j+1) *(x(i)-x0(j+1) ) -p(j+2)*( x(i) -x0(j) ) )... %行列式
    %             /( x0(j) -x0(j+1)  )
    %     end
    %设计数据结构
    p=zeros(1,m);q=zeros(1,m);%赋初值
    q=y0; %赋值
    for t=1:m-1
        p=q;          %重新把q存放到p上
        for j=t:m-1
            q(j+1)=( p(t) *(x(i)-x0(j+1) ) -p(j+1)*( x(i) -x0(t) ) )... %行列式
```

```
                /( x0(t) -x0(j+1)  );
        end
    end
    y(i)=q(m);
end
end
```

newton_llh2.m

```
function y=newton_llh2(x0,y0,x)
%x0为插值点，y0为插值点，x为近似点，
m=length(x0);
n=length(x);
%b=zeros(1:m+1);
for l=1:n        %%%我们输入的是数组
    xx=x(l);     %%%近似点赋值
    for k=1:m    %%%m--->n
        b=0;   %修改的地方!
        for i=1:k
            a=1;
            %        b=0 错误的地方!
            for j=1:k  %%%
                if j~=i
                    a=a*(x0(i)-x0(j));
                end

            end
            b=b+y0(i)/a;%得出f[x1,x2,x3,...,xk]
        end

        p(k)=b;%p(k)=f[x1,x2,x3,...xk]
    end
    s=p(1);

    for k=2:m
        sg=1;
        for j=1:(k-1)
            sg=sg*(xx-x0(j));          %得出每一个项f[x1,x2,x3...,xk]*(x-x1)*(x-x2)*(x-x3)...
        end
        s=s+p(k)*sg;%算出整体的f（x）
    end
    y(l)=s;
end
end
```

newton.m

```
function s=newton(x,y,t)
%定义符号变量
syms p;  % 这种做法调用maple内核，运算缓慢，不支持使用
```

```
s=y(1);
xishu=0;
dxs=1;
%读取x的长度
n=length(x);
%%%%%%%%%%%%%%%%
%%构造牛顿插值方法
for(i=1:n-1)
    for(j=i+1:n)
        xishu(j) = (y(j)-y(i))/( x(j)-x(i) );
    end
    temp1(i)=xishu(i+1);
    dxs=dxs*(p-x(i));
    s=s+temp1(i)*dxs;
    y=xishu;
end
simplify(s);
%%%%%%%%%%%%%%
m=length(t);
for i=1:m
    temp(i)=subs(s,'p',t(i) );
end
s=temp;
end
```

10.2 列选主元的Gauss消去法

num_course140415.m

```
clear
clc
n=10;
A=hilb(n);
b=ones(n,1);
x=linemain_cxf(A,b,n);
rx=norm(b-A*x)
y=A\b;
ry=norm(b-A*y);
```

linemain_cxf.m

```
function [b] = linemain_cxf(A,b,n)
% 列主元高斯消去法
% 1. 输入系数矩阵A, 右端项b, 阶n。

% 2. 对 k=1,2,...,n-1,循环:
for k = 1 : n-1
    % (a) 按列选主元
    %\alpha : = max_{k<=i<=n} |a_{ik}| ,
    alpha = max(abs(A(k:n,k)));
```

```
    % 保存主元所在行的指标i_k
    for j = k : n
        if( abs( A(j,k) ) == alpha );
            i_k = j;
            break;
        end
    end
    % (b) 若\alpha = 0, 则系数矩阵奇异, 计算停止; 否则, 顺序进行。
    if(alpha == 0)
        disp('系数矩阵奇异! ');
        return;
    end
    % (c) 若i_k = k则转向(d);否则换行
    %                a_{i_k,j} <-----> a_{k,j},  j=1,...,n
    %                  b_{i_k} <-----> b_k
    if( i_k ~= k )
        temp = A(k,1:n);
        A(k,1:n) = A(i_k,1:n);
        A(i_k,1:n) = temp;
        bb = b(k);
        b(k) = b(i_k);
        b(i_k) = bb;
    end
    % (d) 计算乘子 m_{i,k} = a_{i,k}/a_{k,k}, i=k+1,...,n.
    for i = k+1 : n
        m(i,k) = A(i,k)/A(k,k);
    end
    % (e) 消元:
    for i = k+1 : n
        for j = k+1 : n
            A(i,j) = A(i,j) - m(i,k)*A(k,j);
        end
        b(i) = b(i) - m(i,k)*b(k);
    end
end
% 3. 回代
for i = n : -1: 1
    if(i<n)
        s = A(i,i+1:n)*b(i+1:n);
    else
        s=0
    end
    b(i) = ( b(i) - s )/A(i,i);
end


end
```